Sorbonne Université

Sciences et Ingénierie

MSc Computer Science – MIND

Large Language Models

Recherche d'information, RAG, Splade

Antoine Théologien

21400184



Encadrant :	Benjamin Piwowarski
Matière :	Large Language Models
Date :	28 septembre 2026

1 Introduction

La recherche d'information (RI) est un domaine en pleine expansion visant à identifier les documents les plus pertinents pour une requête donnée. Largement dominé par des approches comme BM25, le RAG (Retrieval Augmented Generation) [Lewis et al., 2020], s'inscrit comme une approche nouvelle combinant LLM et RI. Un contexte est alors fourni à un LLM afin d'améliorer les résultats. Dans ce rapport, nous nous intéressons à la mise en place d'une pipeline de RAG en utilisant SPLADE (Sparse Lexical And Expansion) [Formal et al., 2021], en effectuant en plus quelques expérimentations afin de comprendre au mieux le modèle et son efficacité. Pour cela, nous nous appuierons sur le dataset LoTTE (Technology).

2 Méthodologie

2.1 Données et Protocole

Nous utilisons le dataset **LoTTE** (Technology). Pour l'entraînement, nous avons généré des triplets via une stratégie de Hard Negative Mining : pour chaque requête, les négatifs sont sélectionnés parmi les documents que BM25 classe faussement comme pertinents (lexicalement proches mais sémantiquement faux). L'indexation et l'évaluation sont réalisées avec la librairie **PyTerrier**.

2.2 Modèles Étudiés

Nous avons implémenté et comparé les variantes suivantes :

- **BM25 (Baseline) :** Approche lexicale standard.
- **SPLADE Std :** Entraînement complet d'un modèle DistilBERT avec régularisation FLOPS pour la sparsité.
- **SPLADE LoRA :** Utilisation de l'adaptation de bas rang (*Low-Rank Adaptation*) [Hu et al., 2022] sur les matrices Query et Value pour réduire le coût mémoire du fine-tuning.
- **Approches par Distillation :** La distillation consiste à utiliser un modèle **Teacher** (plus performant) pour guider l'apprentissage de notre modèle **Student** (SPLADE). Le Teacher est un Cross-Encoder qui prend en entrée la paire (Requête, Document) et capture les interactions fines. Le Student apprend à imiter la distribution de scores du Teacher via une perte MSE (Mean Squared Error). Nous avons testé deux Teachers :
  - **Distill (Std) :** Un Cross-Encoder entraîné localement sur nos données.
  - **Distill++ (Expert) :** Un Cross-Encoder MS-MARCO pré-entraîné, beaucoup plus robuste.

2.3 Métriques d'évaluation

- **MRR (Mean Reciprocal Rank) :** Mesure la précision du premier résultat pertinent.
- **Recall@k (R@k) :** Mesure la proportion de documents pertinents retrouvés dans le Top-k.

3 Premiers Résultats : Entraînement Local

Dans cette première partie, nous analysons les performances des modèles entraînés localement sur le dataset restreint.

TABLE 1 – Comparatif des modèles entraînés localement vs Baseline

Metric	**BM25** (Baseline)	**SPLADE** (Std)	**LoRA** (Peft)	**Distill** (StdTeach)	**Distill++** (Expert)
MRR	**0.8067**	0.5488	0.2655	0.0289	0.0875
Recall@1	**0.3367**	0.1642	0.0458	0.0000	0.0100
Recall@5	**0.7250**	0.3842	0.1992	0.0100	0.0325
Recall@10	**0.8333**	0.4983	0.2950	0.1017	0.0992

BM25 s'avère être une baseline extrêmement forte sur ce dataset technique. Le modèle SPLADE parvient à capturer une partie du signal (MRR 0.54) mais reste globalement moins performant.

Les méthodes de distillation et LoRA ont souffert d'un manque de données, menant potentiellement à un phénomène de dimension collapse, probablement dû à la difficulté d'apprendre la sparsité et la sémantique simultanément sur un petit volume. Nous avions tout de même augmenté la quantité de données d'entraînement et fait varier le nombre d'epochs et le learning rate, sans grand succès.

4 Analyse de l'Expansion par Chain-of-Thought (CoT)

Pour tenter d'améliorer le rappel du modèle SPLADE Std, nous avons utilisé un LLM (`SmolLM2`) pour générer des sous-requêtes explicatives avant la recherche, en nous basant sur une approche de Chain-of-Thought [Wei et al., 2022]. Nous avons ainsi distingué deux paramètres pouvant éventuellement influencer la réussite ou non de cette méthode : la température et le nombre de sous-requêtes générées. Nous avons ainsi étudié l'influence de chacun de ces paramètres sur la performance de notre modèle.

4.1 Impact de la Température

La température est un paramètre qui contrôle la créativité du LLM. Si elle est élevée, le modèle générera des sous-requêtes plus originales mais aussi plus imprévisibles, et donc le contraire si la valeur de température est petite. On étudie ainsi l'impact de ce paramètre en prenant des valeurs comprises entre 0.1 et 1.2, avec un pas de 0.2. Le résultat est visible dans la figure 1.

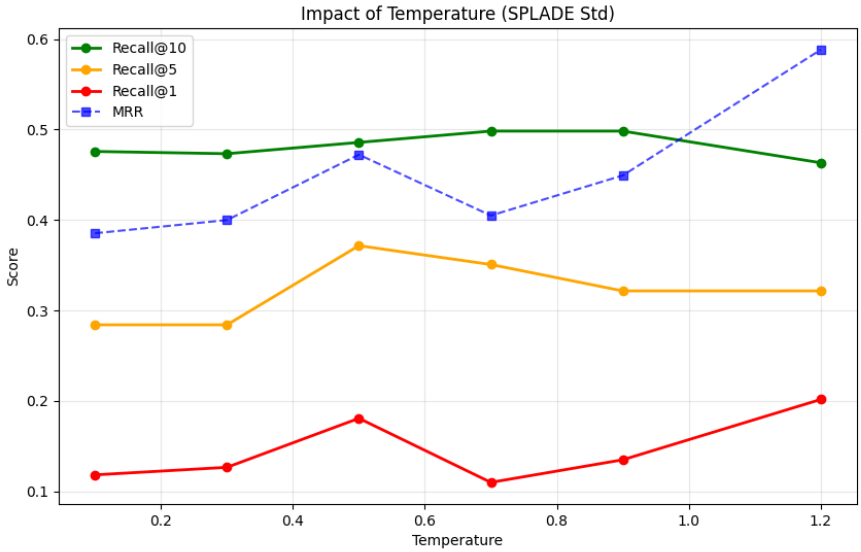


FIGURE 1 – Évolution des métriques en fonction de la température.

On observe un pic de performance lorsque la température est assez élevée, le MRR et le Recall@1 étant alors à leur valeur les plus élevée. On remarque cependant qu'une température de 0.5 semble être un bon compromis, permettant d'avoir un MRR semblable au résultat original, ainsi que des valeurs de Recall@k globalement à leurs pics.

Il faut cependant relativiser ces résultats : la température apportant des éléments aléatoires, les résultats peuvent changer en fonction de l'imprévisibilité du LLM, une température élevée pouvant alors aussi réduire les résultats.

4.2 Impact de la Quantité de Requêtes (N)

On va donc maintenant s'intéresser à la quantité de sous-requêtes générées. L'objectif est de voir si le fait de multiplier les reformulations permet de mieux couvrir le sujet et d'augmenter le rappel. En théorie, plus on a de variantes, plus on a de chances de tomber sur de bons mots-clés.

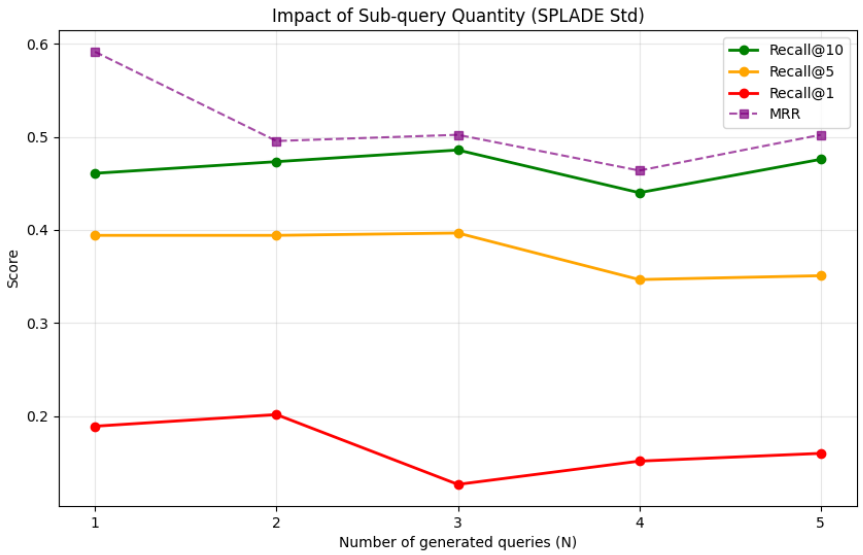


FIGURE 2 – Performance en fonction du nombre de sous-requêtes générées.

On observe ainsi qu'augmenter N a tendance à réduire la précision, sans grande influence sur les valeurs de rappel. Une génération d'une seule sous-requête semble donc être la stratégie la plus efficace, le modèle semblant ensuite se perdre et diluer l'information pertinente.

5 Approche Pré-entrainée et Hybridation

Face aux limites de l'entraînement local sur peu de données, nous avons décidé d'intégrer un modèle Splade pré-entrainé, afin de bénéficier de l'efficacité prouvée de SPLADE. Nous avons ainsi mis en place une stratégie d'hybridation entre ce modèle et BM25, afin de voir si combiner les deux pouvait éventuellement améliorer les résultats.

Cela permettrait de bénéficier à la fois de l'approche de BM25 sur les mots exacts et l'approche de SPLADE portant sur la sparsité. Pour cela, on combine linéairement les scores normalisés des deux modèles :

$$S_{\text{final}} = \alpha \cdot S_{\text{BM25}} + (1 - \alpha) \cdot S_{\text{SPLADE}}$$

5.1 Optimisation du paramètre α

On étudie alors l'influence du coefficient α, afin de trouver une éventuelle valeur permettant de garder le meilleur des deux modèles, et donc d'avoir une hybridation efficace. La Figure 3 présente ainsi l'évolution du MRR en fonction de la valeur de ce coefficient.

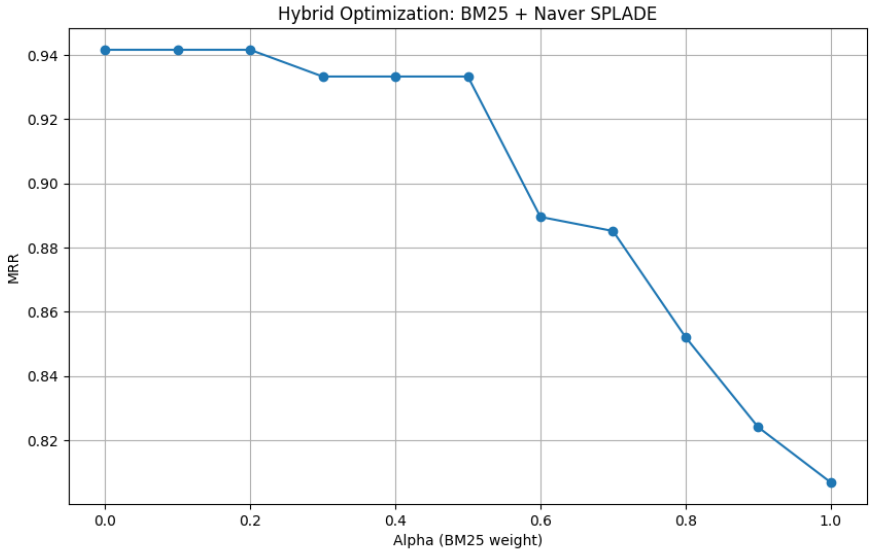


FIGURE 3 – Optimisation du paramètre Alpha pour l'hybridation (BM25 + Splade Préentrainé).

On remarque ainsi que l'optimum du coefficient α se situe entre 0 et 0.2, BM25 prenant ensuite une place trop importante et réduisant alors les résultats.

5.2 Résultats Finaux

On compare alors nos résultats obtenus avec notre baseline BM25. Pour l'approche hybride, on a ici $\alpha \approx 0.2$.

TABLE 2 – Comparatif Final : Approches Pré-entraînées et Hybrides

Metric	**BM25** (Rappel)	**Naver SOTA** (Pre-trained)	**Hybride** (BM25 + Naver)
MRR	0.8067	**0.9417**	**0.9417**
Recall@1	0.3367	**0.4242**	**0.4242**
Recall@5	0.7250	**0.8675**	**0.8675**
Recall@10	0.8333	**0.8842**	**0.8842**

6 Analyse Qualitative : Le RAG en action

Pour aller au-delà de simples métriques, nous voulons observer la qualité des réponses générées par notre pipeline de RAG en fonction du modèle. Nous comparons alors les réponses produites par le modèle SPLADE local et le modèle pré-entraîné.

6.1 Exemple 1 : Concept Théorique

Requête : *"Why FPGA is faster than CPU ?"*

— **SPLADE Std (Local) :** Explique correctement la différence entre le traitement séquentiel (CPU) et le traitement parallèle (FPGA). Le modèle a réussi à retrouver un document

expliquant le concept de clock tick et d'exécution simultanée.

— **Splade Pre-trained :** Fournit une réponse très similaire, mais ajoute des détails sur les opérations logiques comme les décalages de bits.

Sur des questions conceptuelles, le modèle local semble être équivalent au modèle pré-entraîné. Il a bien capturé la sémantique de la requête et fourni un contexte pertinent au LLM.

6.2 Exemple 2 : Problème Technique Précis

Requête : *"How to calculate battery percentage using Arduino ?"*

— **SPLADE Std (Local) :** Propose une approche théorique. Il donne la formule mathématique :

$$\text{Voltage} = \frac{\text{measured}}{1024} \times V_{bat}$$

C'est une réponse correcte mais générique.

— **Splade Pre-trained :** Le modèle pré-entraîné a retrouvé un contexte contenant du code (mention de `analogRead(A0)`). Résultat : le LLM génère directement un bloc de code prêt à l'emploi pour l'Arduino.

C'est ici que l'écart de MRR rentre en jeu. Le modèle pré-entraîné est plus précis pour retrouver des passages exacts (comme des bouts de code), là où le modèle local retrouve des documents pertinents mais plus généralistes. En revanche, cela prouve que notre modèle SPLADE local est fonctionnel : il fournit tout de même de bonnes réponses, mais fonctionne moins bien que le modèle pré-entrainé.

7 Conclusion

En conclusion, ce projet nous a montré qu'entraîner un modèle SPLADE à partir de zéro est une tâche complexe, surtout avec un jeu de données limité. On a vu que nos modèles locaux (Std, LoRA, Distillation) peinent à battre la baseline BM25, qui reste très performante sur du vocabulaire technique strict.

Par contre, dès qu'on passe sur un modèle pré-entraîné (SOTA), les performances explosent avec un MRR qui monte à 0.94. L'analyse qualitative a aussi prouvé que même si les scores de notre modèle local sont plus bas, il permet quand même au RAG de générer des réponses cohérentes, mais moins précises techniquement. Finalement, on remarque que l'hybridation peut éventuellement permettre de bénéficier des avantages du BM25 et de SPLADE combinés.

Références

[Formal et al., 2021] Formal, T., Piwowarski, B., and Clinchant, S. (2021). Splade : Sparse lexical and expansion model for first stage ranking.

[Hu et al., 2022] Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., and Chen, W. (2022). Lora : Low-rank adaptation of large language models. In *International Conference on Learning Representations*.

[Lewis et al., 2020] Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Kuttler, H., Lewis, M., Yih, W.-t., Rocktäschel, T., Riedel, S., and Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive nlp tasks. In *Advances in Neural Information Processing Systems*.

[Wei et al., 2022] Wei, J., Wang, X., Schuurmans, D., Bosma, M., Chi, E., Le, Q., and Zhou, D. (2022). Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems*.